

网络安全——

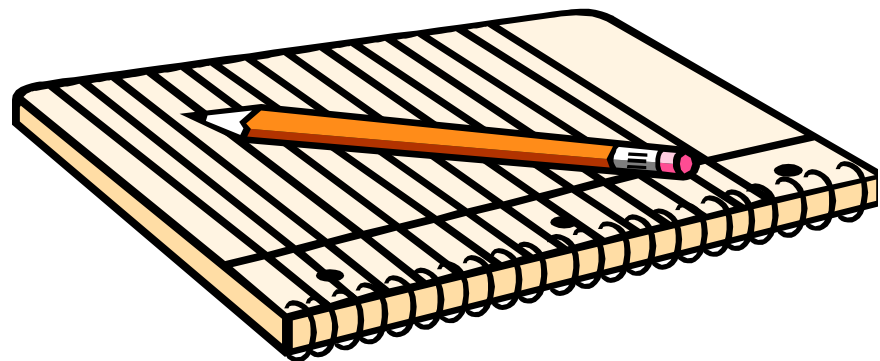
入侵检测技术

北京邮电大学 网络空间安全学院

武 斌

binwu@bupt.edu.cn

本次课程内容（入侵检测）



● 入侵检测概述

● 入侵检测结构

● 入侵检测技术

● 入侵检测部署

● 入侵检测示例

怎么做检测？

- 假如你是一个新闻类门户网站的安全主管，你怎么检测网络攻击威胁？

可能的攻击：

- DoS类
 - SQL注入、跨站等
 - 挂马、篡改类
 - ...
- 如果是一个公司内部服务，又有什么不同？

怎么做攻击检测？

- 为什么要检测？
- 是什么要检测？
- 凭什么来检测？
- 用什么来检测？
- 能检测出什么？
- **检测的意义**
- **检测的目的**
- **检测的依据**
- **检测的方法**
- **检测的结果**

检测的目的和意义

- 并不是所有威胁都要检测
 - 有些威胁无法检测，如搭线窃听
 - 有些威胁没必要检测，如物理攻击
- 检测是为了发现和保护
 - 传统的各类攻击发现：DoS、扫描等
 - 非授权用户的威胁：伪造、伪装等
 - 基本功能的保证：量子密钥协商
 - 复杂攻击的分析：APT

检测的依据

- 攻击的参与者
 - 攻击者
 - 攻击手段
 - 攻击目标
 - 可能被利用的第三方
- 对应检测依据
 - 攻击者信息：IP、账号等
 - 攻击过程：网络数据、日志等
 - 攻击目标信息：IP、服务等
 - 可能被利用的第三方：关系

IDS的历史

- **IDS: intrusion detection system, 入侵检测系统:**
 - 概念的诞生: 1980年4月, James P. Anderson 为美国空军做了题为《计算机安全威胁监控与监视》的技术报告, 第一次详细阐述了入侵检测的概念, 将威胁分为外部渗透、内部渗透和不法行为三种;
 - 模型的发展: 1984-1986年, Dorothy Denning 和 Peter Neumann 研究出了一个实时入侵检测系统, 取名为: **IDES** (入侵检测专家系统);
 - 里程碑的产生: 1990年, 加州大学的L.T.Heberlein等人开发出了 **NSM** (network security monitor)。该系统第一次直接将网络流作为审计数据来源; 从此入侵检测系统发展史形成了两大阵营: 基于网络的**IDS**和基于主机的**IDS**。

为什么需要IDS

- 入侵很容易
 - 入侵教程随处可见
 - 各种工具唾手可得
- **防火墙不能保证绝对的安全**
 - 网络边界的设备
 - 自身可以被攻破
 - 对某些攻击保护很弱
 - 不是所有的威胁来自防火墙外部

动态安全模型P2DR



P2DR模型示意图

入侵检测概念

- 入侵检测是**从计算机网络或计算机系统**中的若干关键点搜集信息并对其进行分析，从中发现网络或系统中是否有违反安全策略的行为和遭到袭击的迹象的一种机制。
- IDS , Intrusion Detection System , 入侵检测系统

入侵检测概念

- **一种主动保护自己的网络 and 系统免遭非法攻击的网络安全技术。**
- 它从计算机系统或者网络中收集、分析信息，检测任何企图破坏计算机资源的完整性、机密性和可用性的行为，即查看是否有违反安全策略的行为和遭到攻击的迹象，并做出相应的反应。
- 安全审计中的核心技术之一

入侵检测功能

- **监控、分析用户和系统活动**

- 入侵检测任务的前提条件

- **发现入侵企图或异常现象**

- 入侵检测系统的核心功能
- 包括两个方面，一是入侵检测系统对进出网络或主机的数据流进行监控，看是否存在对系统的入侵行为，另一个是评估系统关键资源和数据文件的完整性，看系统是否已经遭受了入侵。

- **记录、报警和响应**

- 入侵检测系统在检测到攻击后，应该采取相应的措施来阻止攻击或响应攻击。入侵检测系统作为一种主动防御策略，必然应该具备此功能。

- **审计**系统的配置和弱点、评估关键系统和数据文件的完整性等

入侵检测的分类（1）

- **根据原始数据的来源**

- **基于主机的入侵检测系统**：监控粒度更细、配置灵活、可用于加密的以及交换的环境
- **基于网络的入侵检测系统**：视野更宽、隐蔽性好、攻击者不易转移证据

- **根据检测原理**

- **异常入侵检测**：根据异常行为和使用计算机资源的情况检测出来的入侵。
- **误用入侵检测**：利用已知系统和应用程序的弱点攻击模式来检测入侵。

入侵检测的分类（2）

- **根据体系结构**

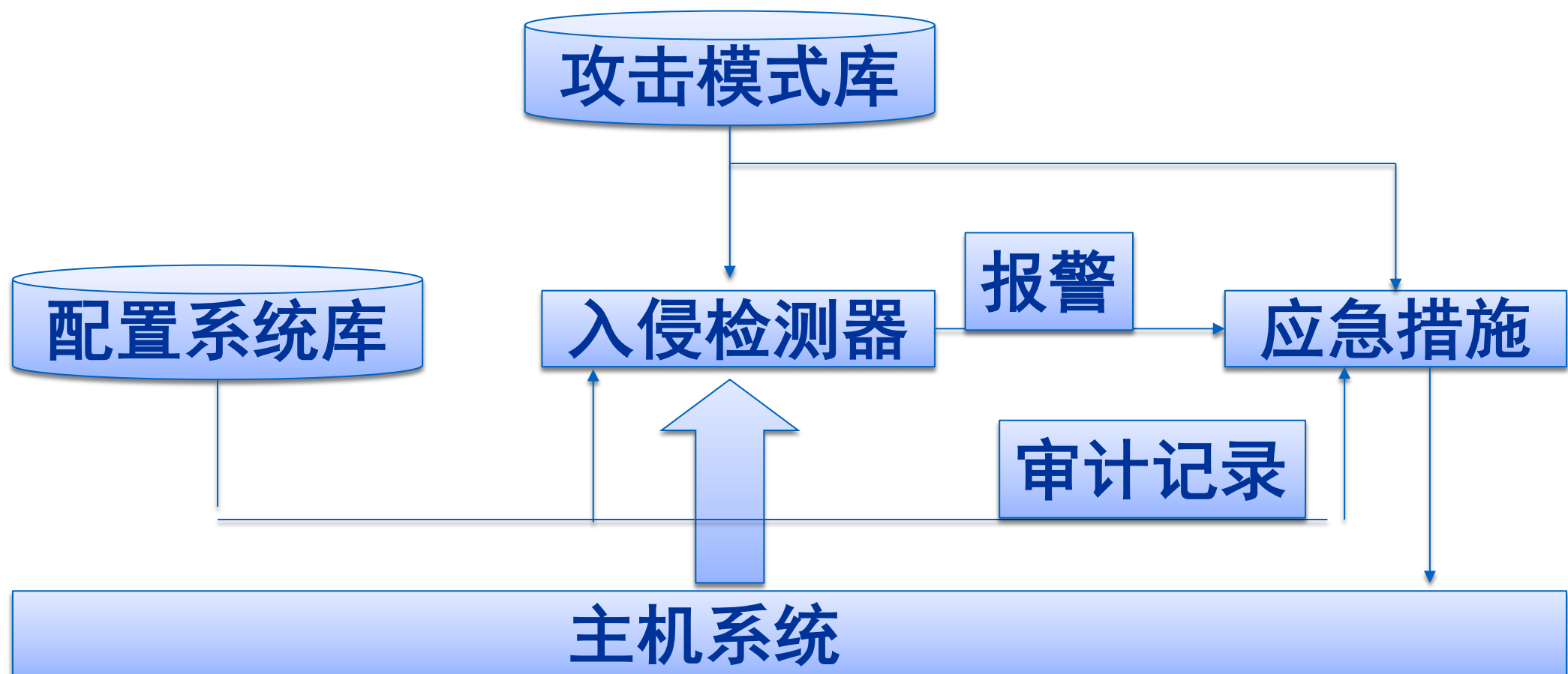
- **集中式**：多个分布于不同主机上的审计程序，一个入侵检测服务器
- **等级式**：定义了若干个分等级的监控区域，每个IDS负责一个区域，然后将当地的分析结果传送给上一级IDS
- **协作式**：将中央检测服务器的任务分配给多个基于主机的IDS，这些IDS不分等级，各司其职，负责监控当地主机的某些活动。

- **根据工作方式分类**

- **离线检测**：非实时工作系统，在事件发生后分析审计事件，从中检查入侵事件。
- **在线检测**：对网络数据包或主机的审计事件进行实时分析，可以快速反应，保护系统的安全；但系统规模较大时，难以保证实时性。

基于主机系统结构

- HIDS : Host-Based IDS
- 检测的目标主要是主机系统和系统本地用户。检测原理是根据主机的审计数据和系统的日志发现可疑事件，检测系统可以运行在被检测的主机或单独的主机上。

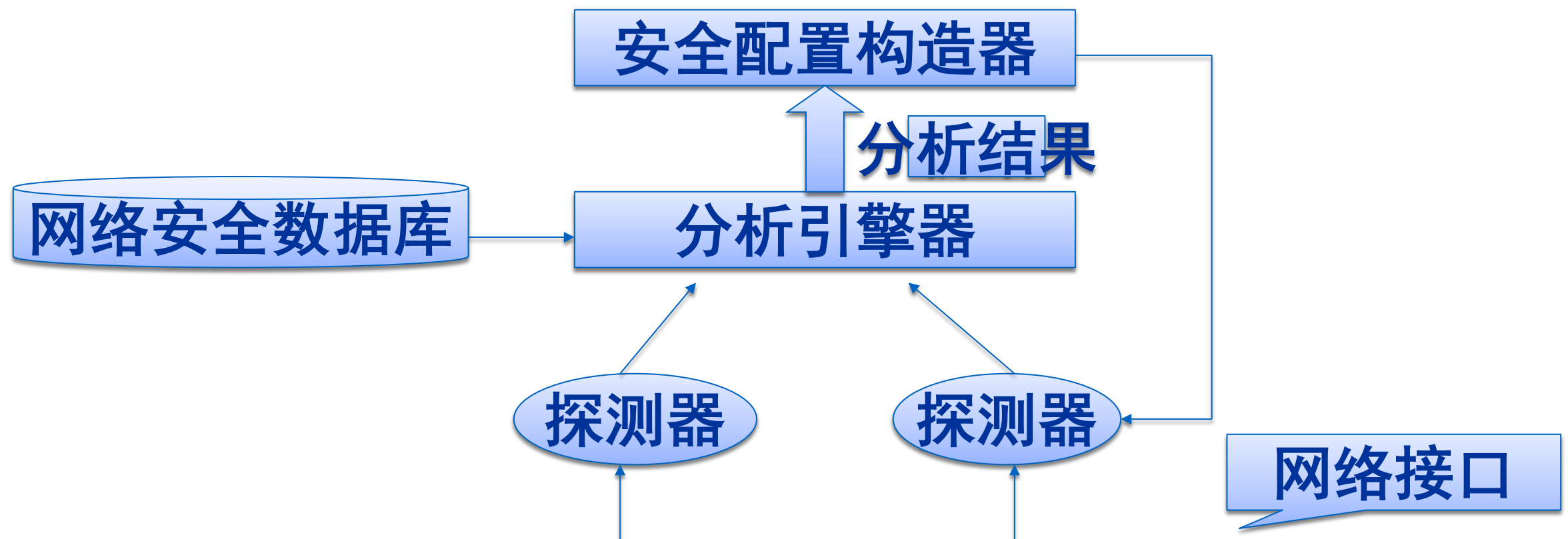


HIDS

- HIDS优点
 - 性能价格比高
 - 细腻性，审计内容全面
 - 视野集中
 - 适用于加密及交换环境
- HIDS缺点
 - IDS的运行影响服务器的性能
 - HIDS依赖性强，依赖于审计数据或系统日志准确性和完整性，以及安全事件的定义。
 - 如果主机数目多，代价过大
 - 不能监控网络上的情况

基于网络系统结构

- NIDS : Network-Based IDS
- 根据网络流量、协议分析、单台或多台主机的审计数据检测入侵。



基于网络系统结构

- NIDS优点：
 - 服务器平台独立性：监视通信流量而不影响服务器的平台的变化和更新；
 - 配置简单：只需要一个普通的网络访问接口即可；
 - 众多的攻击标识：探测器可以监视多种多样的攻击包括协议攻击和特定环境的攻击。
- NIDS缺点
 - 不能检测不同网段的网络包
 - 很难检测复杂的需要大量计算的攻击
 - 协同工作能力弱
 - 难以处理加密的会话

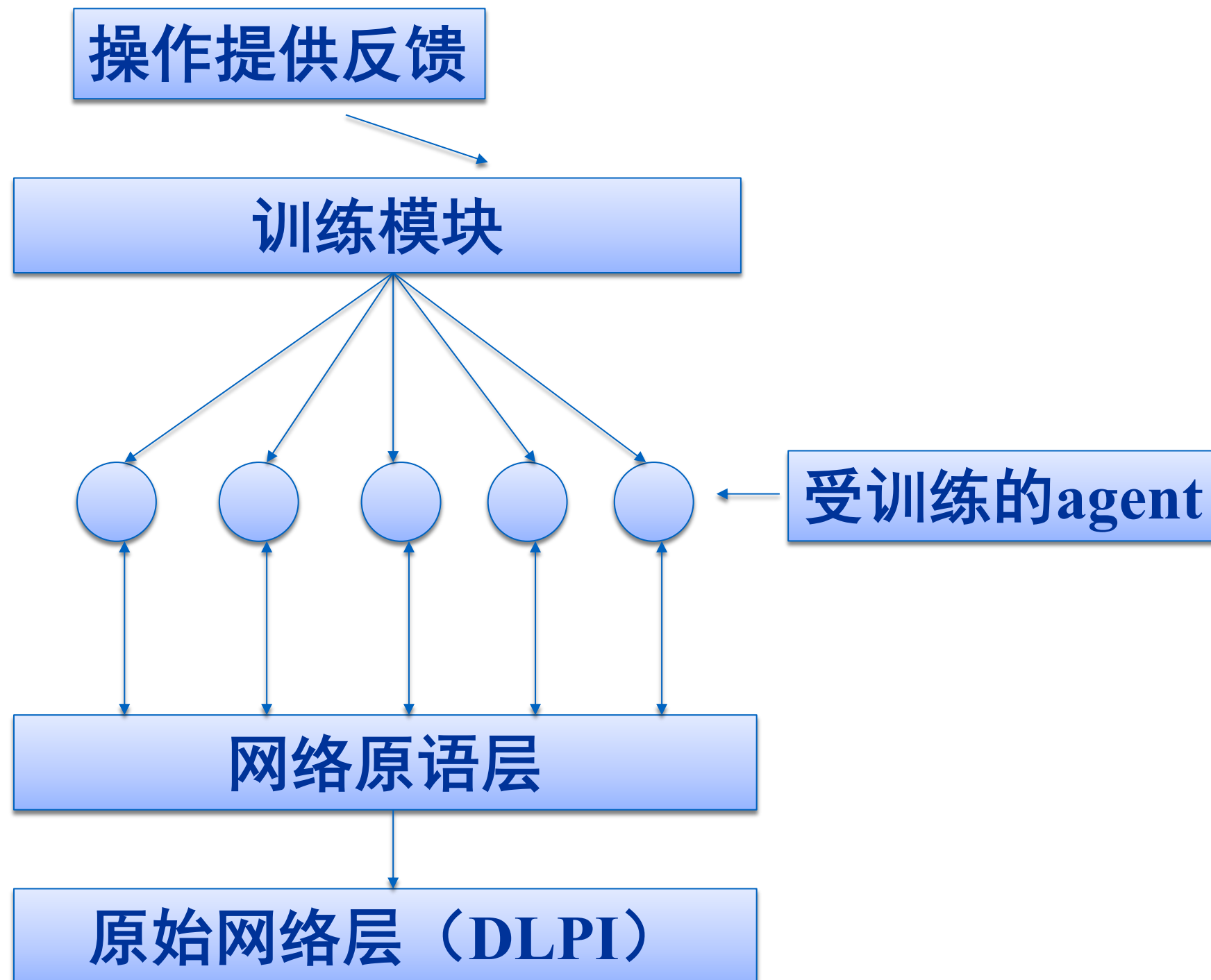
基于分布式系统的结构

- 传统的集中式IDS的基本模型是在网络的不同网段放置多个探测器收集当前网络状态的信息，然后将这些信息传送到中央控制台进行处理分析。
- 分布式结构采用了本地主体处理本地事件，中央主体负责整体分析的模式

基于分布式系统的结构

- 集中式IDS的缺陷：
 - 对于大规模的分布式攻击，中央控制台的负荷将会超过其处理极限，这种情况会造成大量信息处理的遗漏，导致漏警率的增高。
 - 多个探测器收集到的数据在网络上的传输会在一定程度上增加网络负担，导致网络系统性能的降低。
 - 由于网络传输的时延问题，中央控制台处理的网络数据包中所包含的信息只反映了探测器接收到它时网络的状态，不能实时反映当前网络状态。

基于分布式系统的结构



基于分布式系统的结构

- 在最底层是原始网络接口。它提供的接口允许程序传输和接收数据链路层包。
- 网络原语层使用原语把从DLPI接口获取原始网络数据，封装成agent可处理的方式。
- 在agent用于监控系统之前，必须训练到可以对入侵作出正确反应。训练是通过一种反馈机制，操作员考虑是否agent的实际行为接近于给定的流量模式所期望的行为，然后给出训练数据。训练要求agent减小入侵误报数。
- 多agent相互协作：每个agent监控整个网络信息流的一个小的方面。有些情况下，可能需要数个agent一起涵盖可能入侵的所有方面。

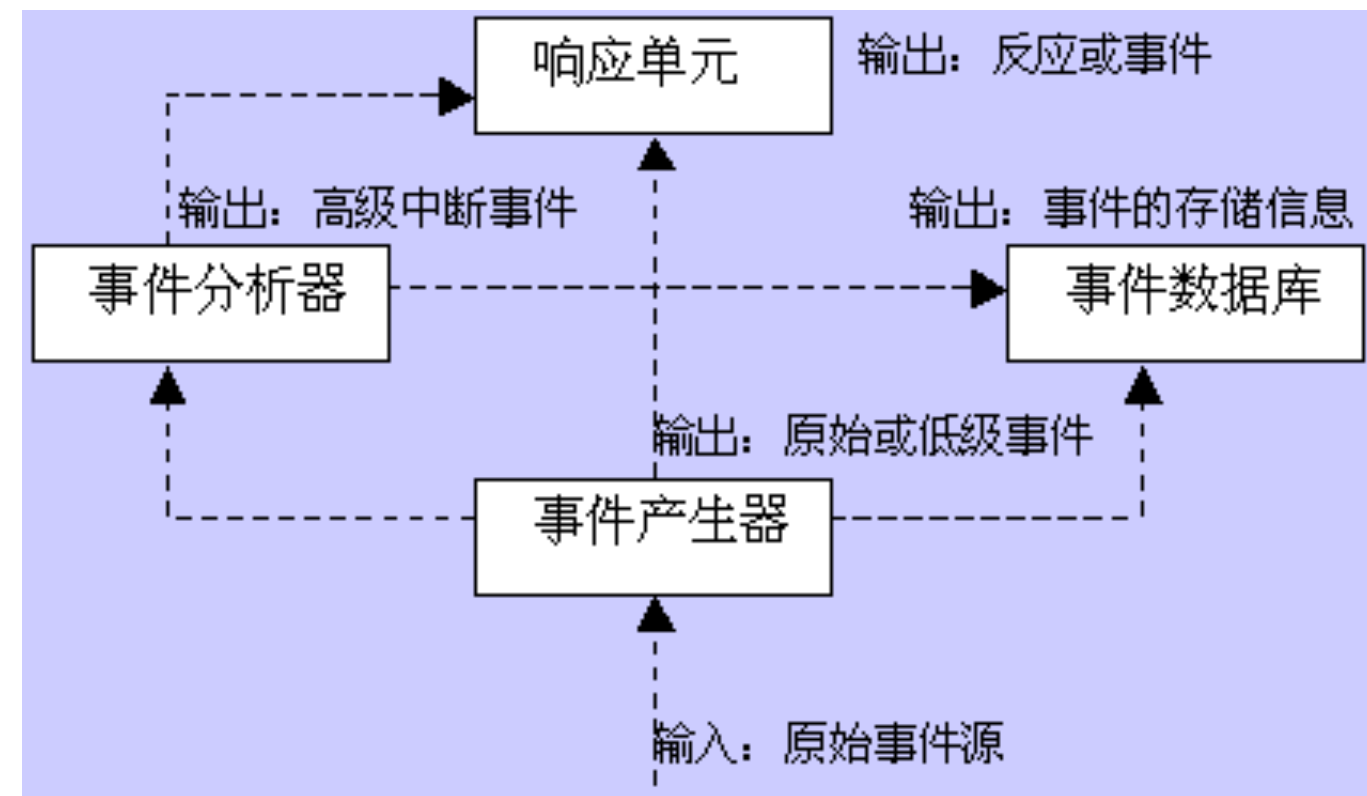
入侵检测技术——

入侵检测结构

入侵检测结构

• CIDEF

- Common Intrusion Detection Framework
- 由DARPA于1997年3月开始着手制定
- 事件产生器(Event Generators)
- 事件分析器(Event analyzers)
- 响应单元(Response units)
- 事件数据库(Event databases)



CIDF

- **事件产生器**

- 事件产生器的目的是从整个计算环境中获得事件，并向系统的其他部分提供此事件。
- 入侵检测的第一步
- 采集内容包括系统日志、应用程序日志、系统调用、网络数据、用户行为、其他IDS的信息

- **事件分析器**

- 事件分析器分析得到的数据，并产生分析结果。
- 分析是核心，效率高低直接决定整个IDS性能

CIDF

- **响应单元**

- 响应单元则是对分析结果作出作出反应的功能单元，功能包括：
 - 告警和事件报告
 - 终止进程，强制用户退出
 - 切断网络连接，修改防火墙设置
 - 灾难评估，自动恢复
 - 查找定位攻击者

- **事件数据库**

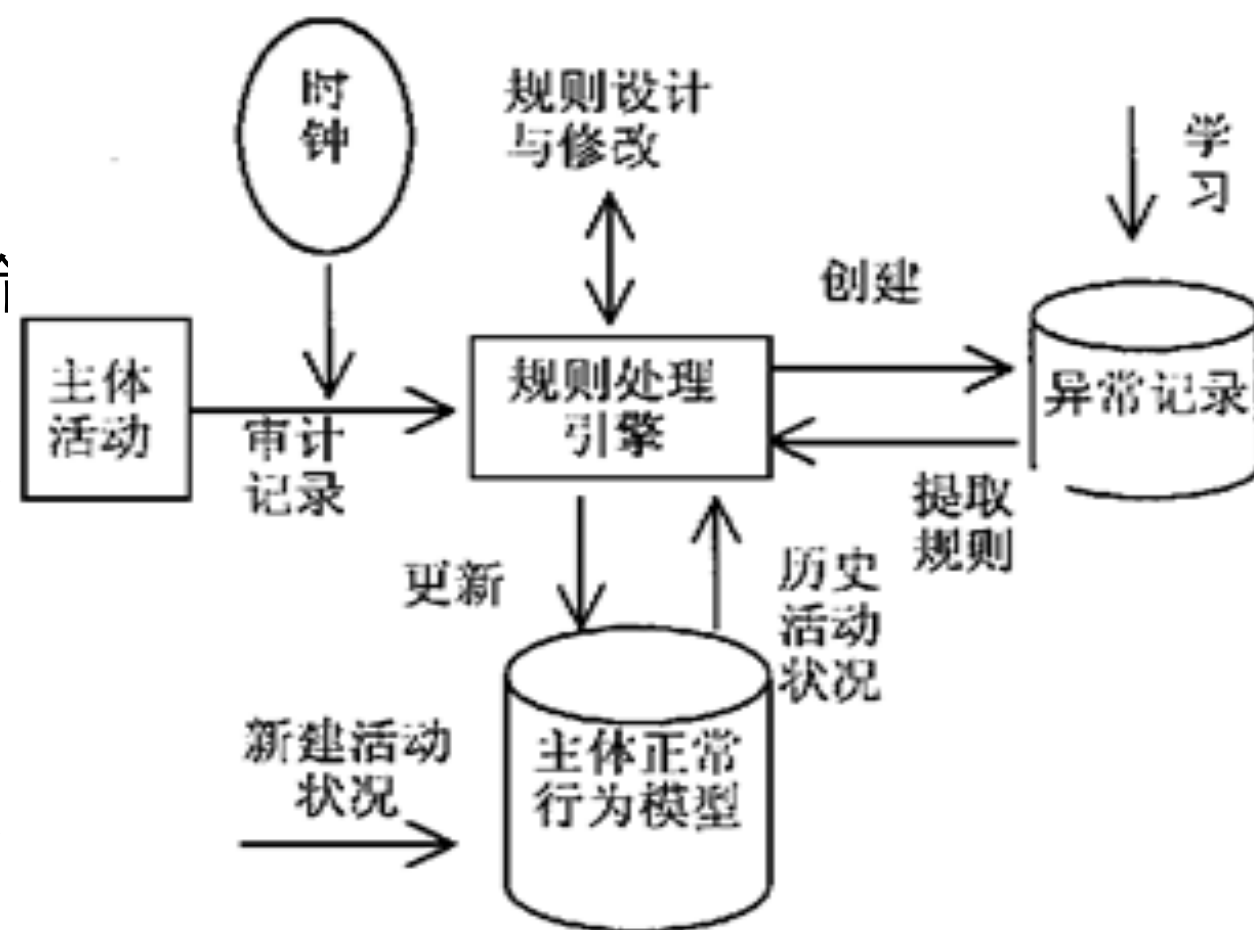
- 事件数据库是存放各种中间和最终数据的地方的统称，它可以是复杂的数据库，也可以是简单的文本文件。

入侵检测结构

• Denning模型

• 1987年提出的一个通用入侵检测模型

- 主体 (Subjects): 在目标系统上活动的实体, 如用户。
- 对象 (Objects): 指系统资源, 如文件、设备、命令等。
- 审计记录 (Audit records): 由主体、活动(主体对目标的操作)、异常条件(系统对主体的该活动的异常情况的报告)、资源使用状况(系统的资源消耗情况)和时间戳(Time-Stamp)等组成。
- 活动档案 (Active Profile): 即系统正常行为模型, 保存系统正常活动的有关信息。
- 异常记录 (Anomaly Record): 由事件、时间戳和审计记录组成, 表示异常事件的发生情况。
- 活动规则 (Active Rule): 判断是否为入侵的准则及相应要采取的行动。



I DWG

- **Intrusion Detection Work Group**

- IDS系统之间、IDS和网管系统之间
- 共享的数据格式
- 统一的通信规程

- **草案**

- IDMEF(入侵检测消息交换格式)
- IDXP (入侵检测交换协议)

攻击分析与攻击检测——

入侵检测技术

异常检测技术

误用检测技术

异常检测技术

- 思想：任何正常人的行为有一定的规律，而入侵会引起用户或系统行为的异常
- 需要考虑的问题：
 - 选择哪些数据来**表现用户的行为**
 - 通过以上数据如何有效地表示用户的行为，主要在于学习和检测方法的不同
 - 考虑学习过程的时间长短、用户行为的时效性等问题
- 典型算法：统计分析（均值、偏差、Markov过程）

异常检测技术

- 异常检测模型（ Anomaly Detection ）：
 - 首先总结正常操作应该具有的特征（ 用户轮廓 ），当用户活动与正常行为有重大偏离时即被认为是入侵
- 检测原理
 - 正常行为的特征轮廓
 - 检查系统的运行情况
 - 是否偏离预设的门限
- 举例
 - 多次错误登录、午夜登录

异常检测技术

- 优点

- 可以检测到未知的入侵
- 可以检测冒用他人帐号的行为
- 具有自适应，自学习功能
- 不需要系统先验知识

- 缺点

- 漏报、误报率高：入侵者可以逐渐改变自己的行为模式来逃避检测；合法用户正常行为的突然改变也会造成误警
- 统计算法的计算量庞大，效率很低
- 统计点的选取和参考库的建立比较困难

异常检测技术

- **统计分析：**

- 依据系统中**特征变量**的历史数据建立统计模型，并运用该模型对特征变量未来的取值进行预测和检测偏离
- 典型的系统主体特征有：系统中的特征变量有用户登录失败次数、CPU和I/O利用率、文件访问数及访问出错率、网络连接数、击键频率、事件间的时间间隔等。
- 缺点：
 - 未考虑事件的发生顺序，所以对利用事件顺序关系的攻击难以检测；
 - 利用统计轮廓的动态自适应性，通过缓慢改变其行为来训练正常特征轮廓，最终使检测系统将其异常活动判为正常；
 - 难以确定门限值。

异常检测技术

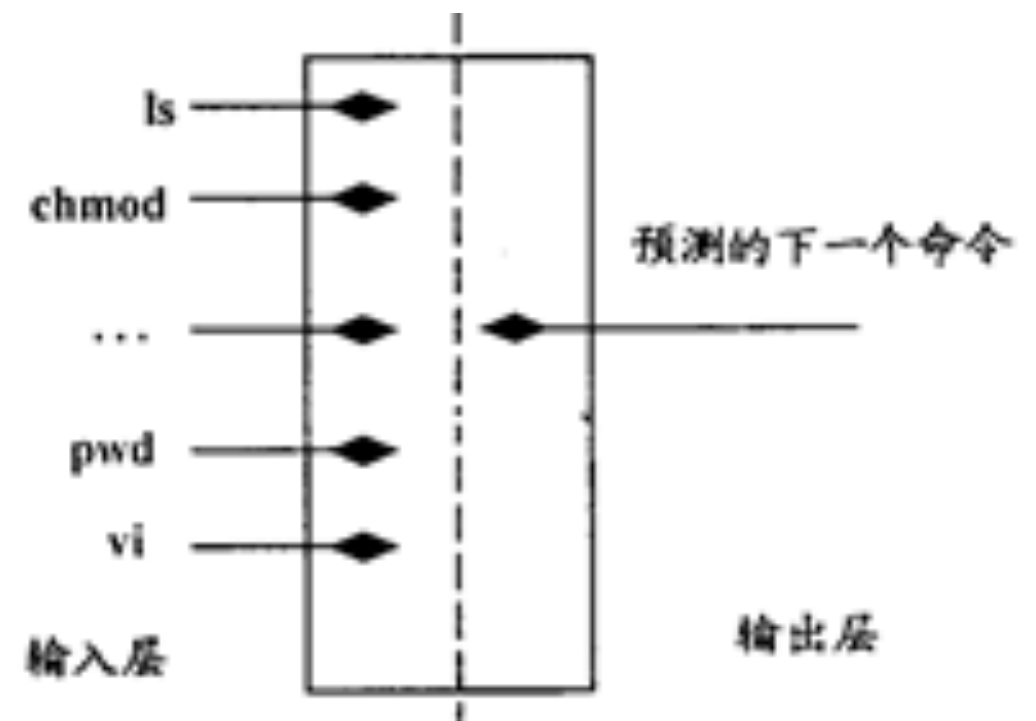
● 统计分析：

- (a) **均值与标准偏差模型**。以单个特征变量为检测对象，假定特征变量满足正态分布，根据该特征变量的历史数据统计出分布参数(均值、标准偏差)，并依此设定信任区间。在检测过程中，若特征变量的取值超出信任区间，则认为发生异常。
- (b) **多元模型**。以多个特征变量为检测对象，分析多个特征变量间的相关性，是均值与标准偏差模型的扩展，不仅能检测到单个特征变量值的偏离，还能检测到特征变量间关系的偏离。
- (c) **Markov过程模型**。将每种类型的事件定义为系统的一个状态，用状态转换矩阵来表示状态的变化，若对应于所发生事件的状态转移概率较小，则该事件可能为异常事件。
- (d) **时间序列模型**。将事件计数与资源消耗根据时间排列成序列，如果某一新事件在相应时间发生的概率较低，则该事件可能为入侵。

异常检测技术

● 神经网络

- 利用神经网络检测入侵的基本思想是用一系列信息单元（命令）训练神经元，这样在给定一组输入后，就可能预测出输出。
- 与统计理论相比，神经网络更好地表达了变量间的非线性关系，并且能自动学习和更新。实验表明UNIX系统管理员的行为几乎全是可以预测的，对于一般用户，不可预测的行为也只占了很少的一部分。



误用检测技术

- 思想：主要是通过某种方式**预先定义入侵行为**，然后监视系统，从中找出符合预先定义规则的入侵行为
- 误用信号需要对入侵的特征、环境、次序以及完成入侵的事件相互间的关系进行描述
- 重要问题
 - 如何全面的描述攻击的特征
 - 如何排除干扰，减小误报
 - 解决问题的方式
- 典型算法：专家系统、模型推理、完整性分析

误用检测技术

- 优点：

- 算法简单
- 系统开销小
- 准确率高
- 效率高

- 缺点：

- 被动：只能检测出已知攻击、新类型的攻击会对系统造成很大的威胁
- 模式库的建立和维护难：模式库要不断更新，知识依赖于硬件平台、操作系统和系统中运行的应用程序等

误用检测技术

● 专家系统

- 将有关入侵的知识转化为if-then结构的规则，即将构成入侵所要求的条件转化为if部分，将发现入侵后采取的相应措施转化成then部分。
- 专家系统主要面临：
 - 1) 全面性问题，即难以科学地从各种入侵手段中抽象出全面地规则化知识；
 - 2) 效率问题，即所需处理的数据量过大，而且在大型系统上，如何获得实时连续的审计数据也是个问题。

误用检测技术

- **模型推理:**

- 模型推理是指**结合攻击脚本推理**出入侵行为是否出现。
- 其中有关攻击者行为的知识被描述为：攻击者目的，攻击者达到此目的的可能行为步骤，以及对系统的特殊使用等。
- 根据这些知识建立攻击脚本库，每一脚本都由一系列攻击行为组成。
- 减少了需要处理的数据量，如何有效地翻译攻击脚本是问题

误用检测技术

- **状态转换分析**

- 将入侵过程看作一个**行为序列**，导致系统从初始状态转入被入侵状态。
- 针对每一种入侵方法确定系统的初始状态和被入侵状态，以及导致状态转换的转换条件
- 用**状态转换图（攻击图）**来表示每一个状态和特征事件
- 状态转换是针对事件序列分析，不善于分析复杂事件，而且不能检测于系统状态无关的入侵。

其它检测技术

- **完整性分析**

- 通过检查系统的当前系统配置，诸如系统文件的内容或者系统表，来检查系统是否已经或者可能会遭到破坏。
- 优点：不管模式匹配方法和统计分析方法能否发现入侵，只要是成功的攻击导致了文件或其它对象的任何改变，它都能够发现。
- 缺点：依赖于完整性数据库，一般以批处理方式实现；非常耗时，不用于实时响应。

其它检测技术

- **计算机免疫技术：**

- 通过正常行为的学习来识别不符合常态的行为序列。
- 程序相对人的行为来说，具有较好的行为稳定性
- 缺点：必须获取所有行为轨迹才能刻画正常行为的轮廓；不能检测出利用合法活动获取非授权存取的攻击

- **数据融合**

- 是针对一个系统中使用多个和 (或)多类的传感器这一特定问题展开的一种新的数据处理方法,因此数据融合又称作多传感器信息融合或信息融合。

其它检测技术

● 遗传算法

- 遗传算法的基本思想来源于Darwin进化论和Mendel的遗传学说，最早由Bagley J.D在1967年提出
- 遗传算法在入侵内检测中的应用时间不长，在一些研究试验中，利用若干字符串序列来定义用于分析检测的指令组，用以识别正常或者异常行为的这些指令在初始训练阶段中不断进化，提高分析能力。
- 也有人将遗传算法与神经网络相结合，将其应用于网络的学习、网络的结构设计和网络的分析，然后应用到入侵检测领域。
- 遗传算法虽然展示了它的潜力和宽广前景，但是它本身还有很多问题需要研究，还存在各种不足，它的应用还有待于进一步的研究。

其它检测技术

- **模糊证据理论**

- 入侵检测的评判标准本身就具有一定的模糊性，模糊证据理论被引入到入侵检测中。
- 建立了基于模糊专家系统的入侵检测框架模型。该模型吸收了滥用检测和异常检测的优点，能较好的降低漏警率和虚警率。

其它检测技术

● 基于数据挖掘的检测方法

- 数据挖掘是一种利用分析工具在大量数据中提取隐含在其中且潜在有用的信息和知识的过程
- 入侵检测过程也是利用所采集的大量数据信息，如系统日志、审计记录和网络数据等，对其进行分析以发现入侵或异常的过程
- 数据挖掘算法有多种：
 - **关联分析**方法主要分析事件记录中数据项间隐含的关联关系，形成关联规则；
 - **序列分析**方法主要分析事件记录间的相关性，形成事件记录的序列模式；
 - **聚类分析**识别事件记录的内在特性，将事件记录分组以构成相似类，并导出事件记录的分布规律。

入侵检测技术——

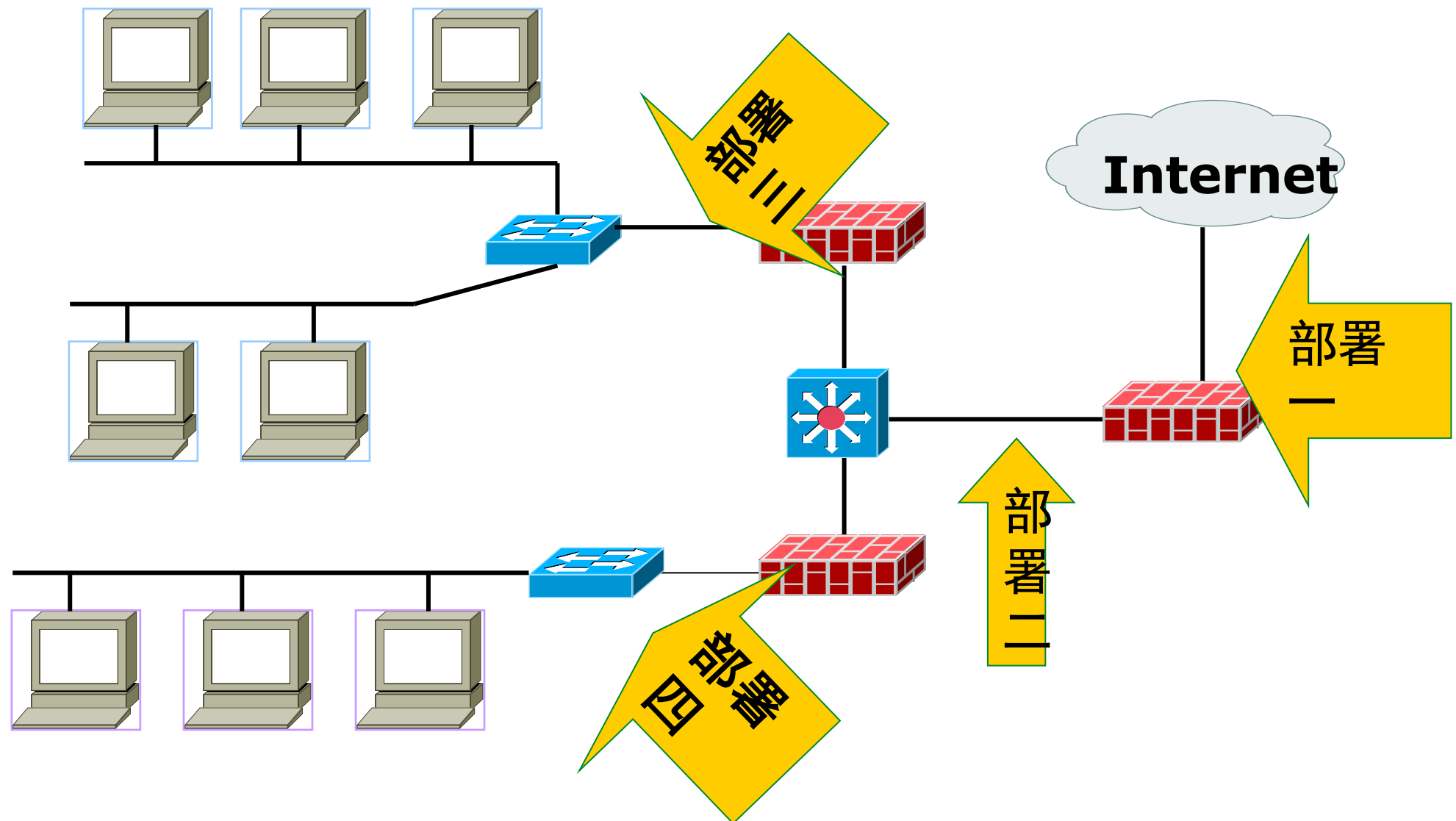
入侵检测部署

IDS部署

- 检测器部署位置
 - 放在边界防火墙之内
 - 放在边界防火墙之外
 - 放在主要的网络中枢
 - 监控大量的网络数据，可提高检测黑客攻击的可能性
 - 放在一些安全级别需求高的子网
 - 对非常重要的系统和资源的入侵检测

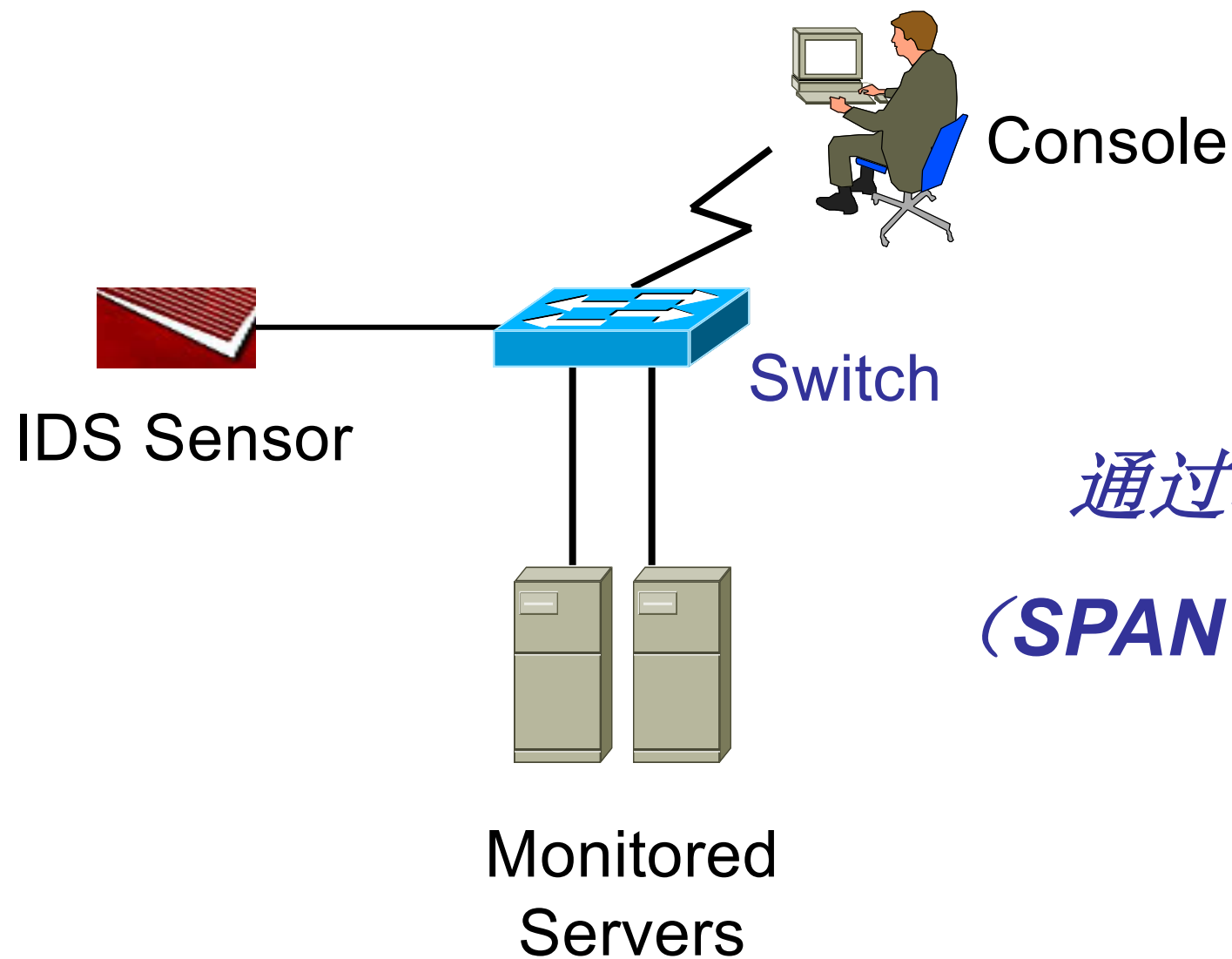
IDS部署

- 检测器部署(一般情况)



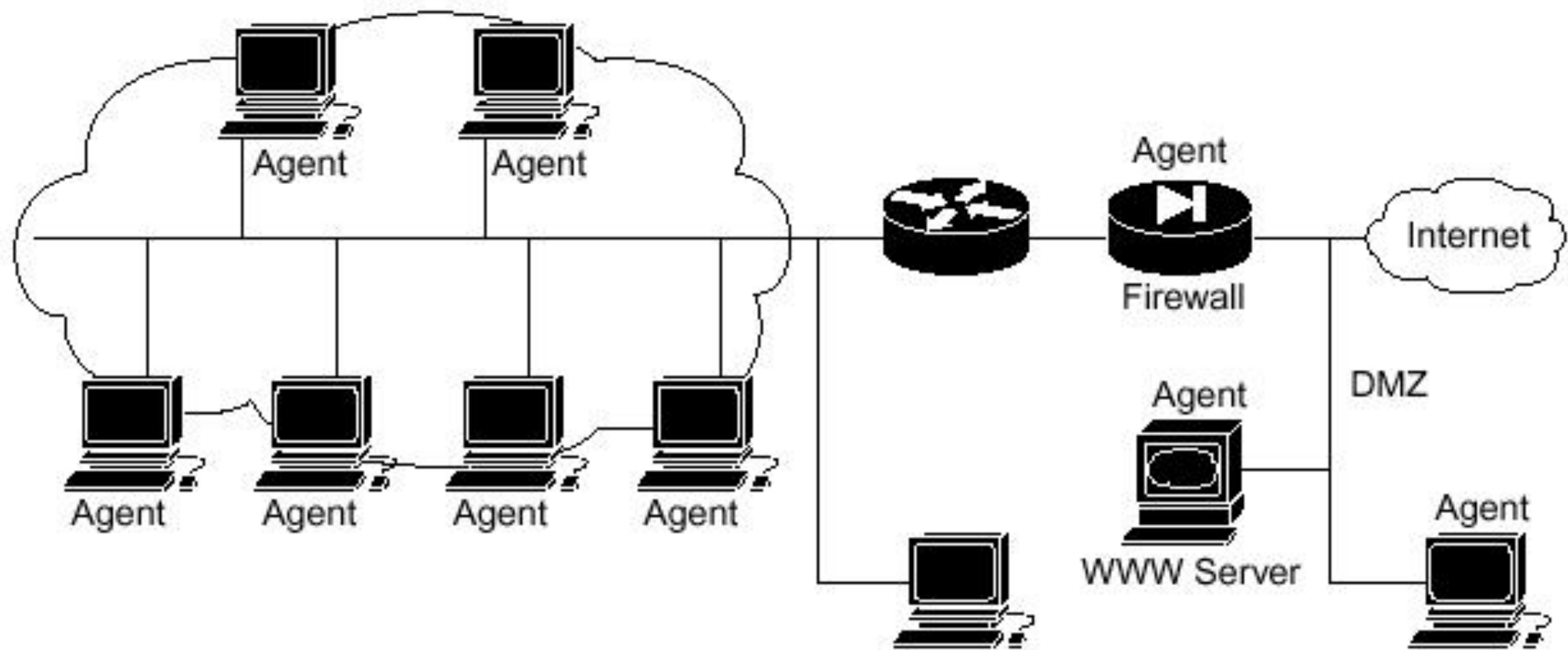
IDS部署

- 检测器部署(交换环境)



通过端口镜像实现
(SPAN / Port Monitor)

IDS部署



入侵检测技术——

入侵检测实例

“Snort”是什么

- Snort是由Sourcefire开发的一个开源的网络入侵检测系统。用C语言编写的开放源代码软件。它是一个跨平台、轻量级的网络入侵检测软件。它结合了协议、特征和基于异常的检测，是目前使用最多的IDS/IPS系统。
- Snort主要**优点**：基于libpcap库调用的网络数据包嗅探和日志记录分析的应用工具，代码开源、功能模块标准化，插件模式使用灵活，运行时相比其他IDS占用资源少、支持多种硬件平台，并能运行在几乎所有的操作系统平台上。



Snort特点

- 典型的误用检测技术
- 需要准确理解攻击模式并撰写检测规则
- 无法应对未知攻击，造成漏报
- 检测效率高、准确率高
- 攻击规则更新要求高，应对攻击方式变化不大的场景

方法：基于攻击特征匹配的原理，准确度高，误报率低，无法检测未知攻击，典型的误用检测技术。

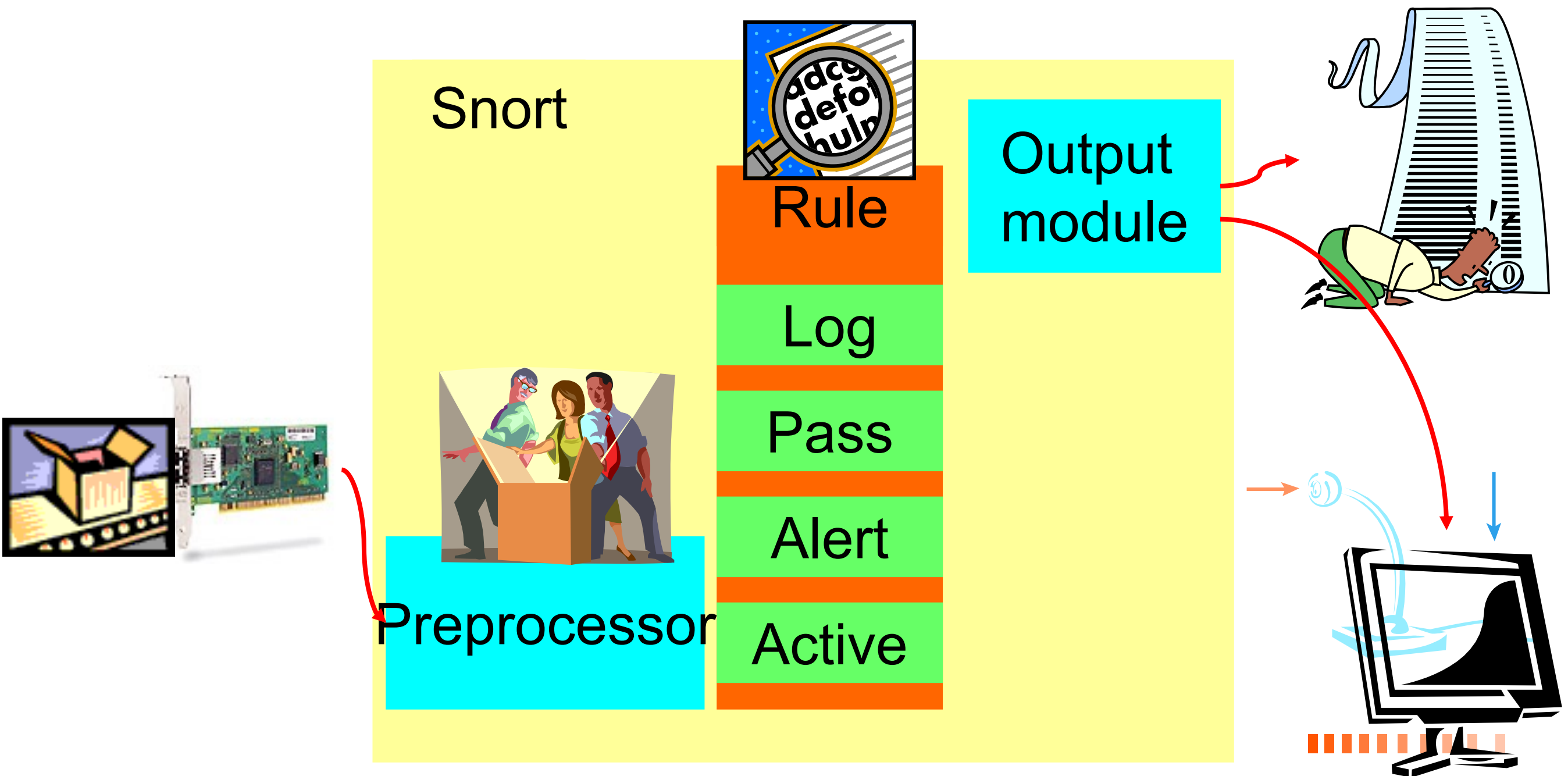
Snort的主要功能

- 截取网络数据报文，进行网络数据实时分析、报警、以及日志的能力。
- 记录网络数据，其日志文件可以是tcpdump格式，也可以是解码的ASCII格式。
- 实时报警能力，可以将报警信息写到syslog、指定的文件、套接字或者使用WinPopup消息。

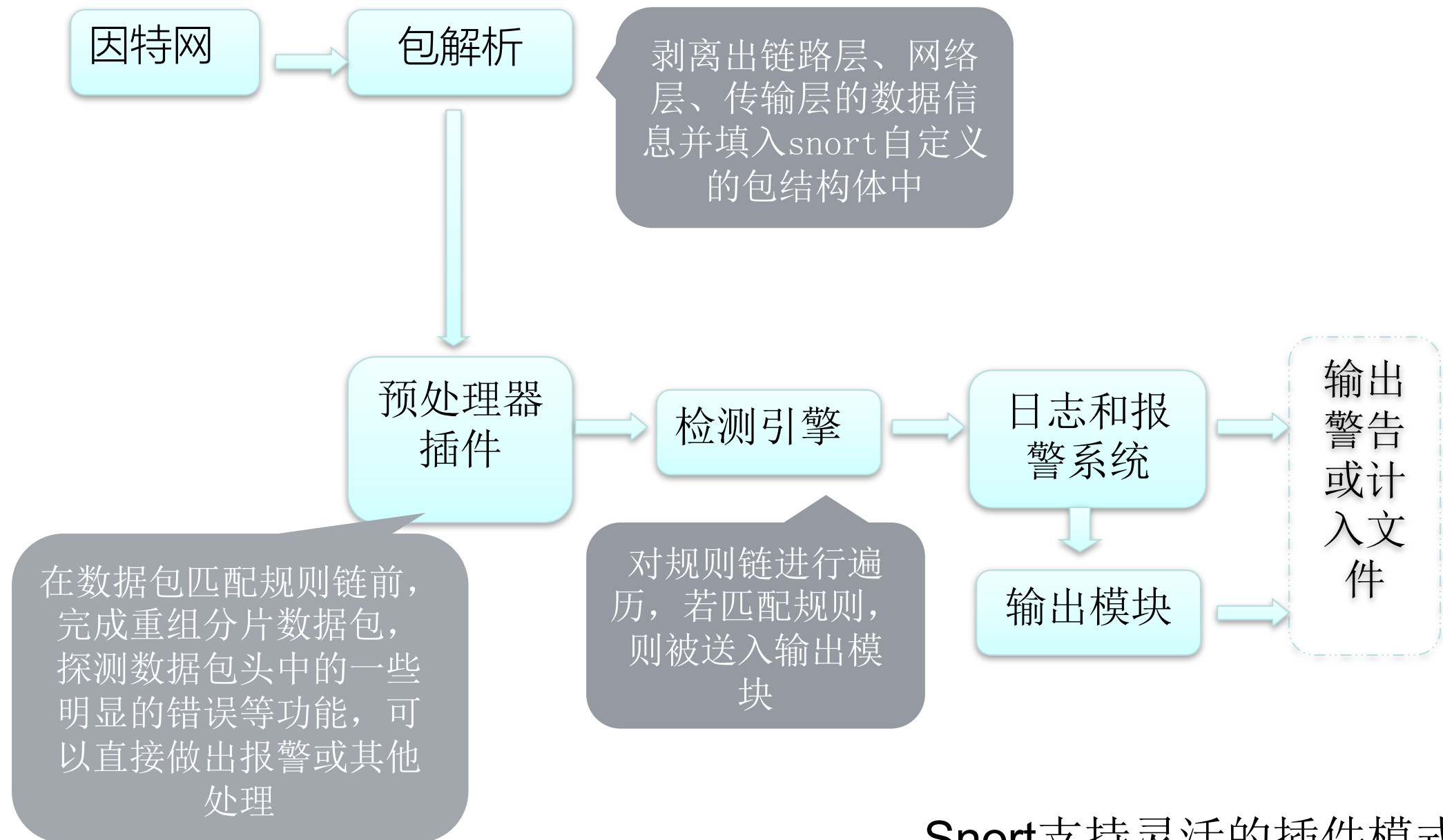
Snort的主要功能 (contd.)

- 能够进行协议分析，内容搜索、匹配，能够用来检测各种攻击和探测，例如：缓冲区溢出、隐秘端口扫描、CGI攻击、SMB探测、OS指纹特征检测等等。
- Snort使用一种灵活的规则语言来描述网络数据报文，因此，可以对新的攻击作出快速地解析。

Snort Working Architecture



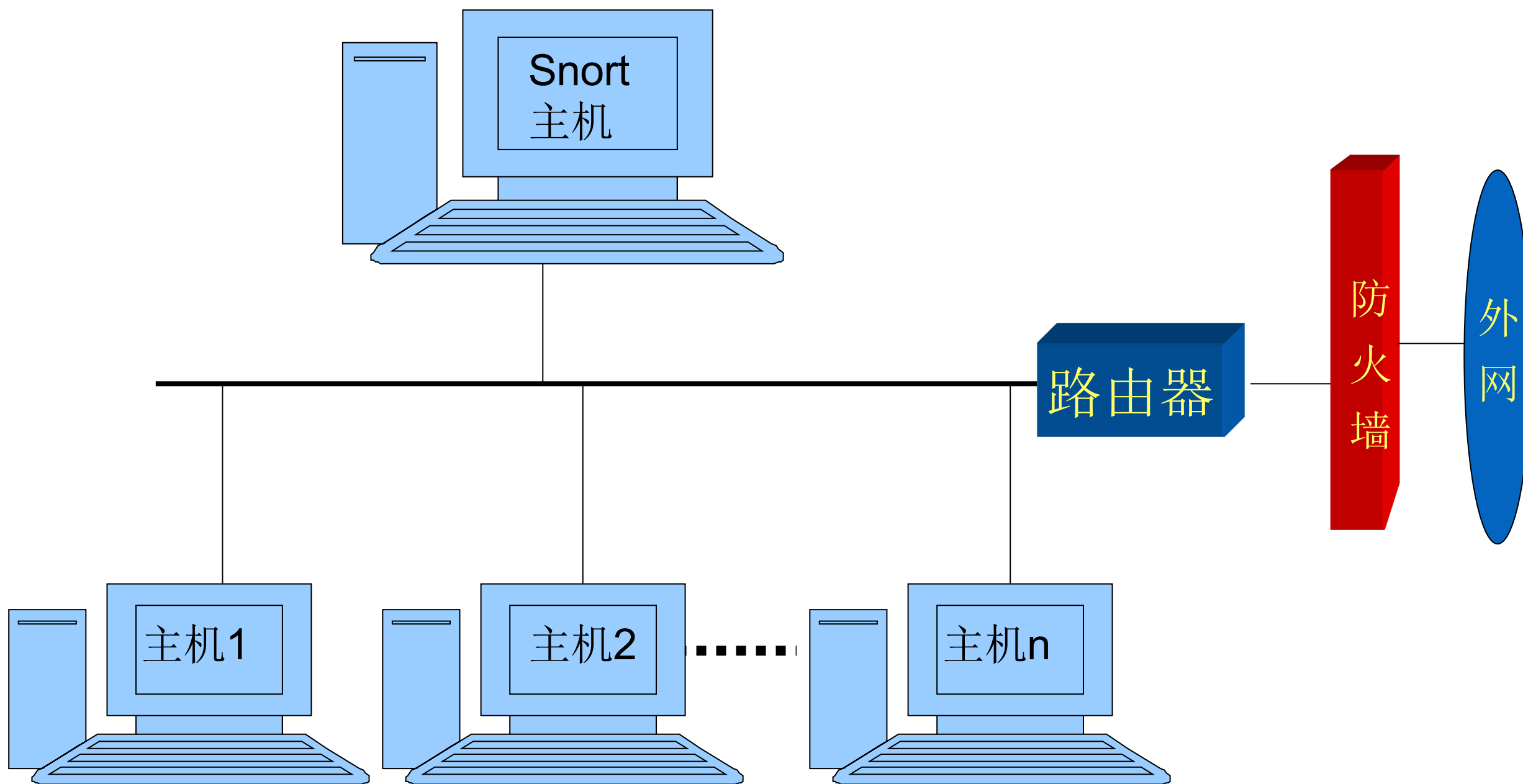
Snort的组成



Snort支持灵活的插件模式

- 预处理插件
- 检测引擎规则插件
- 报警输出插件

Snort的部署



Snort系统监控图

Snort的工作模式

- Sniffer mode: 包嗅探模式，此模式下snort从网卡抓包，并实时显示在控制台。
- Packet logger mode: 包记录模式，此模型下snort从网卡抓包，并记录。
- Network Intrusion Detection mode (NIDS): 入侵检测模式，分析数据包并与入侵规则库相匹配来发现入侵行为。

运行snort 2.9.0.3包嗅探模式于Ubuntu10.04

一个小实例

```
root@moonriver:/etc/snort# snort -vde
Running in packet dump mode
```

```
--== Initializing Snort ==--
```

```
Initializing Output Plugins!
```

```
pcap DAQ configured to passive.
```

```
Acquiring network traffic from "eth0".
```

```
Decoding Ethernet
```

```
--== Initialization Complete ==--
```

```
-*> Snort! <*-
```

```
Version 2.9.0.3 IPv6 (Build 98)
```

```
By Martin Roesch & The Snort Team: http://www.snort.org/snort/snort-team
```

```
Copyright (C) 1998-2010 Sourcefire, Inc., et al.
```

```
Using libpcap version 1.0.0
```

```
Using PCRE version: 8.12 2011-01-15
```

```
Using ZLIB version: 1.2.3.3
```

```
Commencing packet processing (pid=2135)
```

```
03/10-14:49:21.248143 00:26:C7:20:E4:4E -> 01:00:5E:7F:FF:FA type:0x800 len:0xAF
```

```
192.168.0.108:49275 -> 239.255.255.250:1900 UDP TTL:1 TOS:0x0 ID:10489 IpLen:20
```

```
DgmLen:161
```

```
Len: 133
```

```
4D 2D 53 45 41 52 43 48 20 2A 20 48 54 54 50 2F M-SEARCH * HTTP/
31 2E 31 0D 0A 48 6F 73 74 3A 32 33 39 2E 32 35 1.1..Host:239.25
35 2E 32 35 35 2E 32 35 30 3A 31 39 30 30 0D 0A 5.255.250:1900..
53 54 3A 75 72 6E 3A 73 63 68 65 6D 61 73 2D 75 ST:urn:schemas-u
70 6E 70 2D 6F 72 67 3A 64 65 76 69 63 65 3A 49 pnp-org:device:I
6E 74 65 72 6E 65 74 47 61 74 65 77 61 79 44 65 nternetGatewayDe
76 69 63 65 3A 31 0D 0A 4D 61 6E 3A 22 73 73 64 vice:1..Man:"ssd
70 3A 64 69 73 63 6F 76 65 72 22 0D 0A 4D 58 3A p:discover"..MX:
33 0D 0A 0D 0A 3....
```

命令行参数

-v:进入嗅探模式，显示IP和

TCP/UDP/ICMP包头信息

-vd: 除上之外，显示更为详细的应用层数据

-vde: 除上之外，显示更为详细的链路层数据

抓到的UDP包的链路层、IP、应用层数据信息

Snort命令行

入侵检测模式下运行**snort**的一个典型命令行如下：

```
Snort -c /etc/snort/snort.conf -l /var/log/snort -A fast -i eth1
```

- c: 指定**snort**运行入侵检测模式时所需的配置文件**snort.conf**的路径
- l: 指定数据包log日志文件的路径
- A: 指定**snort**输出报警信息的方式，“fast”表示快速输出模式，报警信息包含的内容简单
- i: 指定抓取数据包的网卡

Snort主要由配置文件**snort.conf**和命令行参数控制运行时的各项选项。命令行参数的优先级高于配置文件中的设置。

Snort规则

EXAMPLE

Rule Header `alert tcp $EXTERNAL_NET $HTTP_PORTS -> $HOME_NET any`

Message `msg: "BROWSER-IE Microsoft Internet Explorer
CacheSize exploit attempt";`

Flow `flow: to_client,established;`

Detection `file_data;
content:"recordset"; offset:14; depth:9;
content:".CacheSize"; distance:0; within:100;
pcree:"/CacheSize\s*=\s*/";
byte_test:10,>,0x3fffffff,0,relative,string;`

Metadata `policy max-detect-ips drop, service http;`

References `reference:cve,2016-8077;`

Classification `classtype: attempted-user;`

Signature ID `sid:65535;rev:1;`

Snort的规则

- Rules are similar as packet-filter expressions
- Snort has 4 rule actions
 - **activate** - alert and then turn on another dynamic rule
 - **dynamic** - remain idle until activated by an activate rule , then act as a log rule
 - **alert** - generate an alert using the selected alert method, and then log the packet
 - **pass** - ignore the packet
 - **log** - log the packet

Snort规则

```
alert tcp $EXTERNAL_NET any -> $HOME_NET any (msg:"POLICY VNC server response";  
flow:established; content:"RFB 0"; depth:5; content:".0"; depth:2; offset:7;  
classtype:misc-activity; sid:560; rev:6;)
```

- msg : 报警信息字符串
- flow: 与数据流和数据流向有关的检测选项, “established”表示已建立的TCP流, 其他值还有“to_server”, “to_client”等
- content: 数据包荷载部分匹配的字符串
- depth: content选项修饰符, 从搜索位置的起始区域开始搜索的最大深度
- offset: content选项修饰符, 从数据包荷载处往后一定字节数, 然后再开始搜索
- classtype:规则分类, 见classification.config
- Sid: 每条规则在所属规则分类下的唯一编号, 见sid-msg.map
- rev: 规则版本号

此规则对应的报警信息:

```
03/25-09:26:23.607928 [**] [1:560:6] POLICY VNC server response [**] [Classification:  
Misc activity] [Priority: 3] {TCP} 192.168.0.119:49515 -> 192.168.0.120:5900
```

报警信息中包含有时间戳、gen、sid、rev、msg、classification、priority、协议类型、源地址、源端口、目的地址、目的端口

Snort规则实例一

DoS检测——Land

- 攻击目的：land 攻击是一种使用相同的源和目的主机发送数据包到某台机器的攻击，结果通常使存在漏洞的机器崩溃。
- 攻击原理：一个特别打造的SYN包中的源地址和目标地址都被设置成某一个服务器地址，这时将导致接受服务器向它自己的地址发送SYN—ACK消息，结果这个地址又发回ACK消息并创建一个空连接，每一个这样的连接都将保留直到超时掉。
- 检测方法：**判断网络数据包的源地址和目标地址是否相同。**

注：实例规则为示意性规则，sid非snort官方编号。

Snort规则实例一

```
alert ip $HOME_NET any -> $HOME_NET any  
(msg:"DoS Land attack"; flags:S; flow : stateless  
classtype:attempted-dos; sid:6001; rev:1;sameip)
```

- 过滤筛选ip数据包，来自本地地址的任意端口，发往本地地址的任意端口，在报警和包日志中打印“DoS Land attack”；数据包中**flags字段的值是SYN**。
- 规则类别标识是attempted-dos；snort规则id号为6001；rev是用来识别规则修改的次数，为1。
- **sameip关键字允许规则检测源IP和目的IP是否相等。**

Snort规则实例二

扫描检测——TCP NULL扫描

- 攻击目的：**可以判断目标主机操作系统是windows还是类Unix系统。**
- 攻击原理：根据RFC 793，类Unix系统接收到没有设置任何标志位的数据包，端口关闭的情况下舍弃掉该数据包并且发送一个RST数据包，端口开放的话不会响应；而Windows系统不满足RFC 793，当收到该数据包的情况下，无论端口开放或者关闭，都会响应一个RST数据包给发送方。
- 检测方法：**验证数据包中所有标志位是否都为0。**

Snort规则实例二

```
alert tcp $EXTERNAL_NET any -> $HOME_NET any  
(msg:"SCAN NULL";flags:0; flow : from_client ; classtype:attempted-recon; sid:6002; rev:1;)
```

- 过滤筛选tcp协议的数据包，来自外部网络的任意端口，发往本地的任意端口，并且**tcp包标志位全部为0**，在报警和包日志中打印“SCAN NULL”
- Flow表示规则只应用到来自客户端的TCP数据包；规则类别标识是attempted-recon (**Attempted Information Leak**) ；
- 规则id为6002，规则修改次数为1。

检测方法实例一：snort

漏洞利用检测——SQL注入漏洞

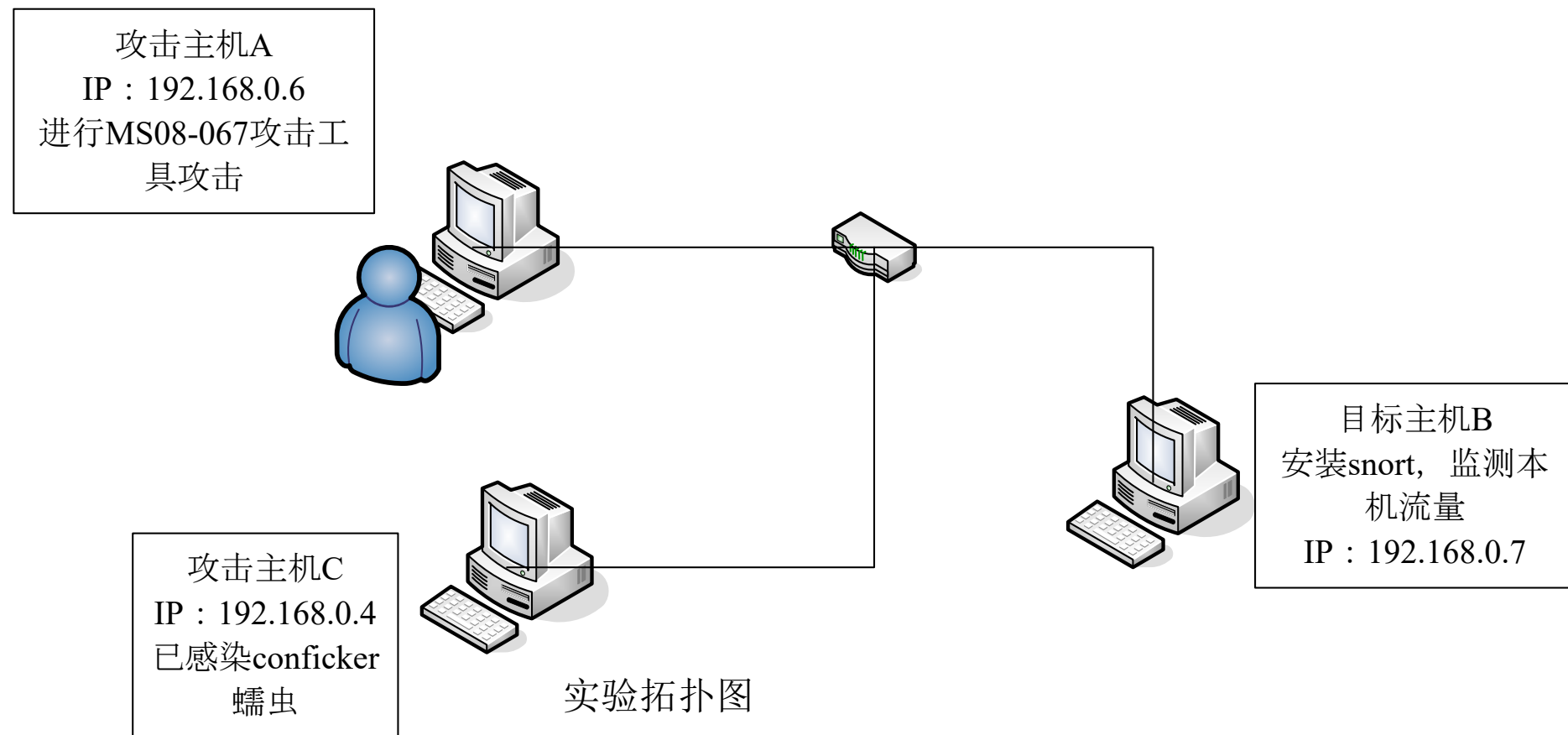
- 攻击目的：恶意用户通过构造特殊字符串从而从服务器获得敏感信息。
- 攻击原理：#可以注释掉SQL语句后面的一行SQL代码，相当于去掉了后面的where条件，而且前面的1=1永远都是成立的，即where子句总是为真，例如select * from users where username="" or 1=1 # and password=md5("")
- 检测方法：验证数据包中的内容中是否存在 “” or 1=1 #”

Snort规则实例三

```
alert tcp $EXTERNAL_NET any -> $HOME_NET any  
(msg:"SQL Injection found"; flow:from_client , established;  
content:""%20or%201=1#"; classtype:web-application-attack;  
sid:6003; rev:1;)
```

- 过滤筛选来自客户端的tcp数据包，来自外部网络的任意端口，发往本地的任意端口，并且包的净荷中含有“**“%20or%201=1#”**”，在报警和包日志中打印“SQL Injection found”
- Flow表示规则只应用到已经建立的来自客户端的TCP连接；
- 规则类别标识是web-application-attack；规则id为6003，规则修改次数为1

基于MS08-067的蠕虫和攻击工具的攻击实验



实验在局域网环境下，三台主机均为windows xp sp3，测试环境中Snort版本为2.9.0.4，采用官网上更新于2011年3月8日的规则库。测试中攻击使用针对windows下MS08-067漏洞的漏洞利用攻击工具和蠕虫。漏洞利用攻击工具的攻击成功的效果是被攻击主机的4444端口被打开，可以远程连接到被攻击主机。蠕虫为conficker蠕虫。

基于MS08-067的蠕虫和攻击工具的攻击实验

实验一：

运行目标主机B上的snort，启动攻击主机A，在A上运用攻击工具metasploit上的MS08-067攻击代码攻击目标主机B

产生的报警信息如下：

```
05/05-20:18:45.192367 [**] [1:17344:1] SHELLCODE x86 OS agnostic xor dword decoder [**]  
[Classification: Executable Code was Detected] [Priority: 1] {TCP} 192.168.0.6:1037 -> 192.168.0.7:445  
05/05-20:18:45.192367 [**] [1:7209:11] NETBIOS DCERPC NCACN-IP-TCP srvsvc NetrPathCanonicalize  
overflow attempt [**] [Classification: Attempted Administrator Privilege Gain] [Priority: 1] {TCP}  
192.168.0.6:1037 -> 192.168.0.7:445
```

实验二：

- (1) 关闭攻击主机A，恢复目标主机B到实验一前的状态，运行B上的snort
- (2) 启动攻击主机C，进行蠕虫攻击

产生的报警信息如下：

```
05/05-20:51:39.894385 [**] [1:7209:11] NETBIOS DCERPC NCACN-IP-TCP srvsvc NetrPathCanonicalize  
overflow attempt [**] [Classification: Attempted Administrator Privilege Gain] [Priority: 1] {TCP}  
192.168.0.4:1041 -> 192.168.0.7:445
```

基于MS08-067的蠕虫和攻击工具的攻击实验

从实验结果的报警信息，可以看到snort报警的特点有：

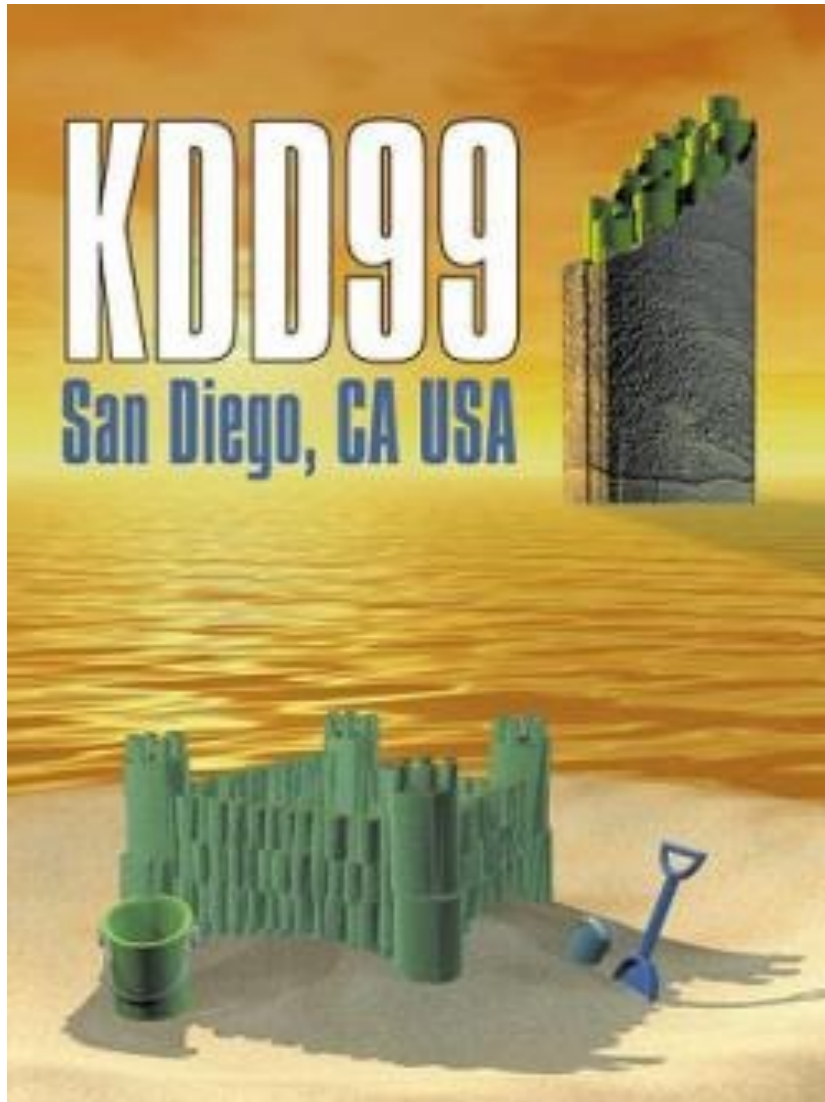
- Snort是对于网络数据包的检测，而杀毒软件则更多的是对于主机数据的监控和检测。
- Snort的报警是基于系统漏洞脆弱性的报警，而不是漏洞利用的过程。

MS08-067漏洞是针对windows xp sp2、sp3、2003 server等系列动态链接库netapi32.dll的NetpwPathCanonicalize函数对于字符串的处理存在典型的缓冲区溢出漏洞，并且NetpwPathCanonicalize可以被RPC调用，漏洞利用成功后可以打开被攻击主机端口并获取权限。

snort可以干什么？

- 根据Snort报警日志，使得网络管理员能够了解目标子网或目标主机的网络环境，并制定相应的防御措施
- 目前Snort支持：
 - (1) 网络中的可疑流量、系统漏洞利用、木马、病毒、蠕虫、后门、拒绝服务攻击、缓冲区溢出攻击、扫描、RPC
 - (2) 支持对于VOIP、SIP、DNS、FTP、TELNET、P2P、ICMP、SMTP、POP、等协议的检测
 - (3) 支持对于sql、web中间件Apache、iis、聊天工具等的检测

检测方法实例二：KDD Cup



- Normal users have rules
- Attacks have rules
- Rules hide in the hosts and network data.
- ML can help us to find these rules from the training data set.

方法：机器学习的方法，需要大量先验数据，样本要全面，针对性的攻击检测难度大。

检测方法实例二：KDD Cup



KDD Cup is the annual Data Mining and Knowledge Discovery competition organized by ACM Special Interest Group on **Knowledge Discovery and Data Mining**, the leading professional organization of data miners. Year to year archives including datasets, instructions, and winners are available for most years.

检测方法实例二：KDD Cup

- KDD-Cup 2008, Breast cancer
- KDD-Cup 2007, Consumer recommendations
- KDD-Cup 2006, Pulmonary embolisms detection from image data
- KDD-Cup 2005, Internet user search query categorization
- KDD-Cup 2004, Particle physics; plus Protein homology prediction
- KDD-Cup 2003, Network mining and usage log analysis
- KDD-Cup 2002, BioMed document; plus Gene role classification
- KDD-Cup 2001, Molecular bioactivity; plus Protein locale prediction.
- KDD-Cup 2000, Online retailer website clickstream analysis
- **KDD-Cup 1999, Computer network intrusion detection**
- KDD-Cup 1998, Direct marketing for profit optimization
- KDD-Cup 1997, Direct marketing for lift curve optimization

KDD Cup 1999

Introduction

KDD Cup 1999: Computer network intrusion detection

The task for the classifier learning contest organized in conjunction with the KDD'99 conference was to learn a predictive model (i.e. a classifier) capable of distinguishing between legitimate and illegitimate connections in a computer network.

KDD Cup 1999 Data

- The 1998 DARPA Intrusion Detection Evaluation Program was prepared and managed by MIT Lincoln Labs.



- **The objective was to survey and evaluate research in intrusion detection.**
- A standard set of data to be audited, which includes a wide variety of intrusions simulated in a military network environment, was provided.
- The 1999 KDD intrusion detection contest uses a version of this dataset.

KDD Cup 1999 Data

- **Lincoln Labs set up an environment to acquire nine weeks of raw TCP dump data for a local-area network (LAN) simulating a typical U.S. Air Force LAN.** They operated the LAN as if it were a true Air Force environment, but peppered it with multiple attacks.
- The raw **training data** was about four gigabytes of compressed binary TCP dump data from **seven weeks** of network traffic. This was processed into **about five million connection records**. Similarly, the **two weeks of test data** yielded around **two million connection records**.

KDD Cup 1999 Data Data Set

- This database contains a standard set of data to be audited, which includes a wide variety of intrusions simulated in a military network environment.

Data Set Characteristic s:	Multivariate	Number of Instances:	4000000	Area:	Computer
Attribute Characteristic s:	Categorical, Integer	Number of Attributes:	42	Date Donated	1999-01-01
Associated Tasks:	Classification	Missing Values?	N/A	Number of Web Hits:	64262

Remark:42 Number of Attributes = 41 kinds of Features + 1 kind of results

<http://www.ll.mit.edu/ideval/data/1999data.html>

<http://kdd.ics.uci.edu/databases/kddcup99/kddcup99.html>

KDD Cup 1999 Data

- The datasets contain a total of 24 training attack types, with an additional 14 types in the test data only.
- Attacks fall into four main categories:
 - **probing**:surveillance and other probing, e.g., port scanning.
 - **DoS**:denial-of-service, e.g. syn flood;
 - **U2R**:unauthorized access to local super user (root) privileges, e.g., various ``buffer overflow" attacks;
 - **R2L**:unauthorized access from a remote machine, e.g. guessing password;

KDD Cup 1999 Data Data Set

PROBE	ipsweep
	mscan
	nmap
	portsweep
	saint
	satan

DoS	apache2
	back
	land
	mailbomb
	neptune
	pod
	processtable
	smurf
	teardrop
	udpstorm

KDD Cup 1999 Data Data Set

U2R	buffer_overflow
	httptunnel
	loadmodule
	perl
	ps
	rootkit
	sqlattack
	xterm

R2L	ftp_write
	guess_passwd
	imap
	multihop
	named
	phf
	sendmail
	snmpgetattack
	snmpguess
	spy
	warezclient
	warezmaster
	worm
	xlock
	xsnoop

Table 1: Basic features of individual TCP connections

Basic features of individual TCP Connections

ID	Feature Name	Description	Type
1	duration	length (number of seconds) of the connection	continuous
2	protocol_type	type of the protocol, e.g. tcp, udp, etc.	discrete
3	service	network service on the destination, e.g., http, telnet, etc.	discrete
4	src_bytes	number of data bytes from source to destination	continuous
5	dst_bytes	number of data bytes from destination to source	continuous
6	flag	normal or error status of the connection	discrete
7	land	1 if connection is from/to the same host/port; 0 otherwise	discrete
8	wrong_fragment	number of ``wrong'' fragments	continuous
9	urgent	number of urgent packets	continuous

Table 2: Content features within a connection suggested by domain knowledge

Content Features Within a Connection Suggested by Domain Knowledge			
ID	Feature Name	Description	Type
10	hot	number of ``hot" indicators	continuous
11	num_failed_logins	number of failed login attempts	continuous
12	logged_in	1 if successfully logged in; 0 otherwise	discrete
13	num_compromised	number of ``compromised" conditions	continuous
14	root_shell	1 if root shell is obtained; 0 otherwise	discrete
15	su_attempted	1 if ``su root" command attempted; 0 otherwise	discrete
16	num_root	number of ``root" accesses	continuous
17	num_file_creations	number of file creation operations	continuous
18	num_shells	number of shell prompts	continuous
19	num_access_files	number of operations on access control files	continuous
20	num_outbound_cmds	number of outbound commands in an ftp session	continuous
21	is_hot_login	1 if the login belongs to the ``hot" list; 0 otherwise	discrete
22	is_guest_login	1 if the login is a ``guest"login; 0 otherwise	discrete

Table 3: Traffic features computed using a two-second time window

Traffic Features Computed Using a Two-second Time Window			
ID	Feature Name	Description	Type
23	count	number of connections to the same host as the current connection in the past two seconds	continuous
		Note: The following features refer to these same-host connections.	
24	serror_rate	% of connections that have ``SYN" errors	continuous
25	rerror_rate	% of connections that have ``REJ" errors	continuous
26	same_srv_rate	% of connections to the same service	continuous
27	diff_srv_rate	% of connections to different services	continuous
28	srv_count	number of connections to the same service as the current connection in the past two seconds	continuous
		Note: The following features refer to these same-service connections.	
29	srv_serror_rate	% of connections that have ``SYN" errors	continuous
30	srv_rerror_rate	% of connections that have ``REJ" errors	continuous
31	srv_diff_host_rate	% of connections to different hosts	continuous

Table 4: Host Features Based on the Purpose of the TCP Connection

Host Features Based on the Purpose of the TCP Connection			
ID	Feature Name	Description	Type
32	dst_host_count	[0, 255]	continuous
33	dst_host_srv_count	[0, 255]	continuous
34	dst_host_same_srv_rate	[0.00, 1.00]	continuous
35	dst_host_diff_srv_rate	[0.00, 1.00]	continuous
36	dst_host_same_src_port_rate	[0.00, 1.00]	continuous
37	dst_host_srv_diff_host_rate	[0.00, 1.00]	continuous
38	dst_host_serror_rate	[0.00, 1.00]	continuous
39	dst_host_srv_serror_rate	[0.00, 1.00]	continuous
40	dst_host_rerror_rate	[0.00, 1.00]	continuous
41	dst_host_srv_rerror_rate	[0.00, 1.00]	continuous

Analysis of KDD99 database

- Learning base : 4 connections have the same 41 attributes but the label is different
- 0,icmp,ecr_i,SF,8,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1,1,0.00,0.00,0.00,0.00,1.00,0.00,0.00,1,1,1.00,0.00,1.00,0.00,0.00,0.00,0.00,0.00,ipsweep.148774
- 0,icmp,ecr_i,SF,8,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1,1,0.00,0.00,0.00,0.00,1.00,0.00,0.00,1,1,1.00,0.00,1.00,0.00,0.00,0.00,0.00,0.00,portsweep.345836
- 0,icmp,tim_i,SF,564,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1,1,0.00,0.00,0.00,0.00,1.00,0.00,0.00,2,2,1.00,0.00,1.00,0.00,0.00,0.00,0.00,0.00,normals.143855
- 0,icmp,tim_i,SF,564,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1,1,0.00,0.00,0.00,0.00,1.00,0.00,0.00,2,2,1.00,0.00,1.00,0.00,0.00,0.00,0.00,0.00,pod.345952

Analysis of KDD99 database

- Test base (corrected file)
 - 71 distinct connections have the same attributes but have the different labels.
 - 71503 (22.99% of the total) connections have the same attributes but appear the different labels
 - 3 ipsweep (Probing) attack connections have the same attributes of those of smurf (DoS) attack (56608 connections)
 - 3 (0.07%) Probing attacks cannot be detected (classified as DoS attack instead)

Analysis of KDD99 database

- Test base (corrected file) :
 - 7563 (97.7% of the total) connections of the snmpgetattack attack have the same attributes of those of normal
 - 2.3% of the snmpgetattack have similar attributes as normal, (but not all the same)
 - 7563 (46.72% of the total) R2L attack cannot be detected (they are classified as normal)

Intrusion detection and Identification based on KDD99 data

- Building the **normal model** based on normal data for intrusion detection
- Building **individual attack model** based on corresponding attack data for intrusion identification

KDD99 数据集检测

- **数据预处理**

- **数据标识**

- 纯文本文件，数据逗号隔开，各个变量没有标识，攻击类别标识统一化

- **数据清洗**

- 存在极个别的不符合规范的数据，需要检查后将其剔除

- **数据抽样**

- 对数据集进行标准化抽样，即按照特定属性对数据进行抽样缩减

- **数据归一化**

- 训练集和测试集中大部分属性的取值范围不同，无法取得相同的离散区间，将取值连续且无固定上下界的数据归一化

- **数据离散化**

- **特征选择**

- **模型评估**

Example:
KNN Based intrusion
detection

k-Nearest Neighbor Classification (kNN)

- Unlike all the previous learning methods, kNN does not build model from the training data.
- To classify a test instance d , define k -neighborhood P as k nearest neighbors of d
- Count number n of training instances in P that belong to class c_j
- Estimate $\Pr(c_j|d)$ as n/k
- No training is needed. Classification time is linear in training set size for each test case.

Basic k-nearest neighbor classification

- Training method:
 - Save the training examples
- At prediction time:
 - Find the k training examples $(x_1, y_1), \dots, (x_k, y_k)$ that are closest to the test example x
 - Predict the most frequent class among those y_i 's.
- Improvements:
 - *Weighting* examples from the neighborhood
 - Measuring “*closeness*”
 - Finding “close” examples in a large training set *quickly*

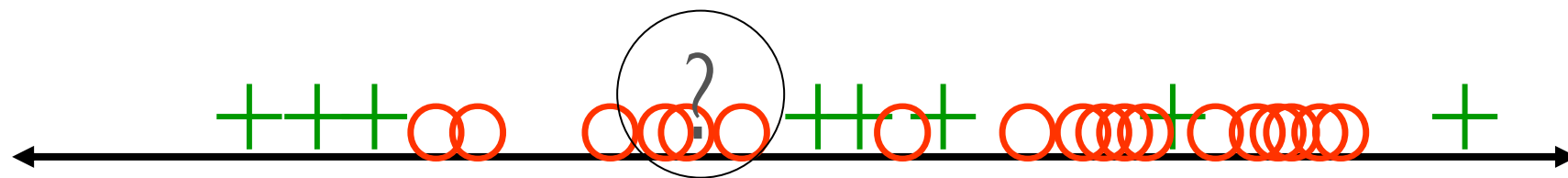
kNN Algorithm

Algorithm $\text{kNN}(D, d, k)$

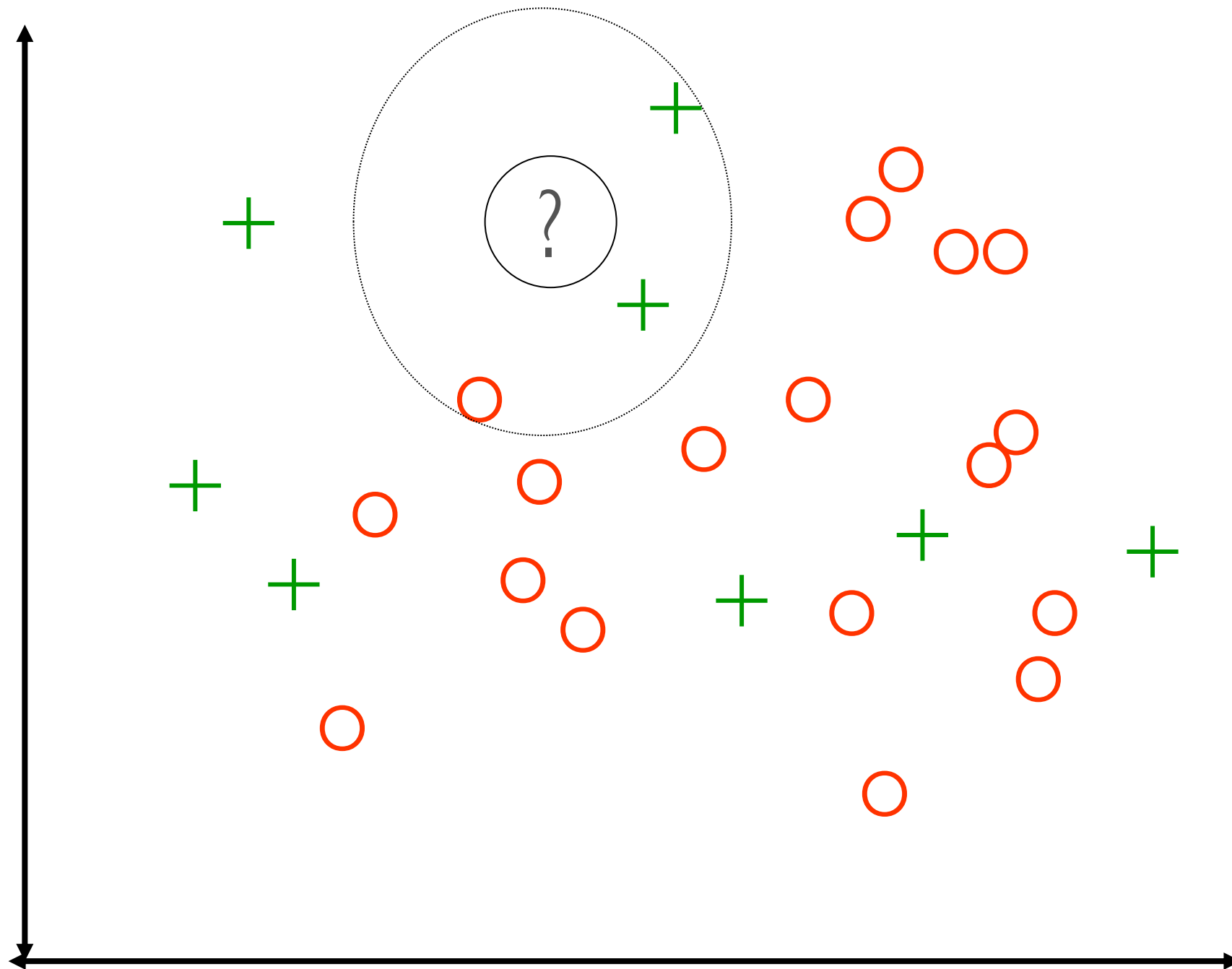
- 1 Compute the distance between d and every example in D ;
- 2 Choose the k examples in D that are nearest to d , denote the set by $P (\subseteq D)$;
- 3 Assign d the class that is the most frequent class in P (or the majority class);

- k is usually chosen empirically via a validation set or cross-validation by trying a range of k values.
- **Distance function** is crucial, but depends on applications.

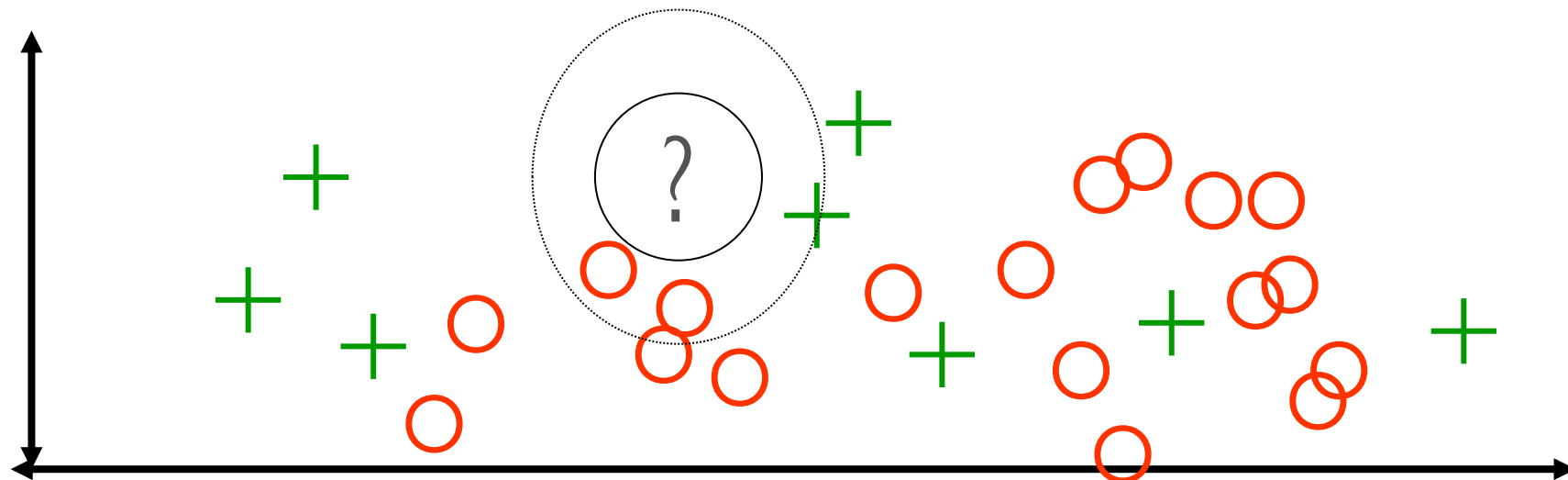
K-NN and irrelevant features



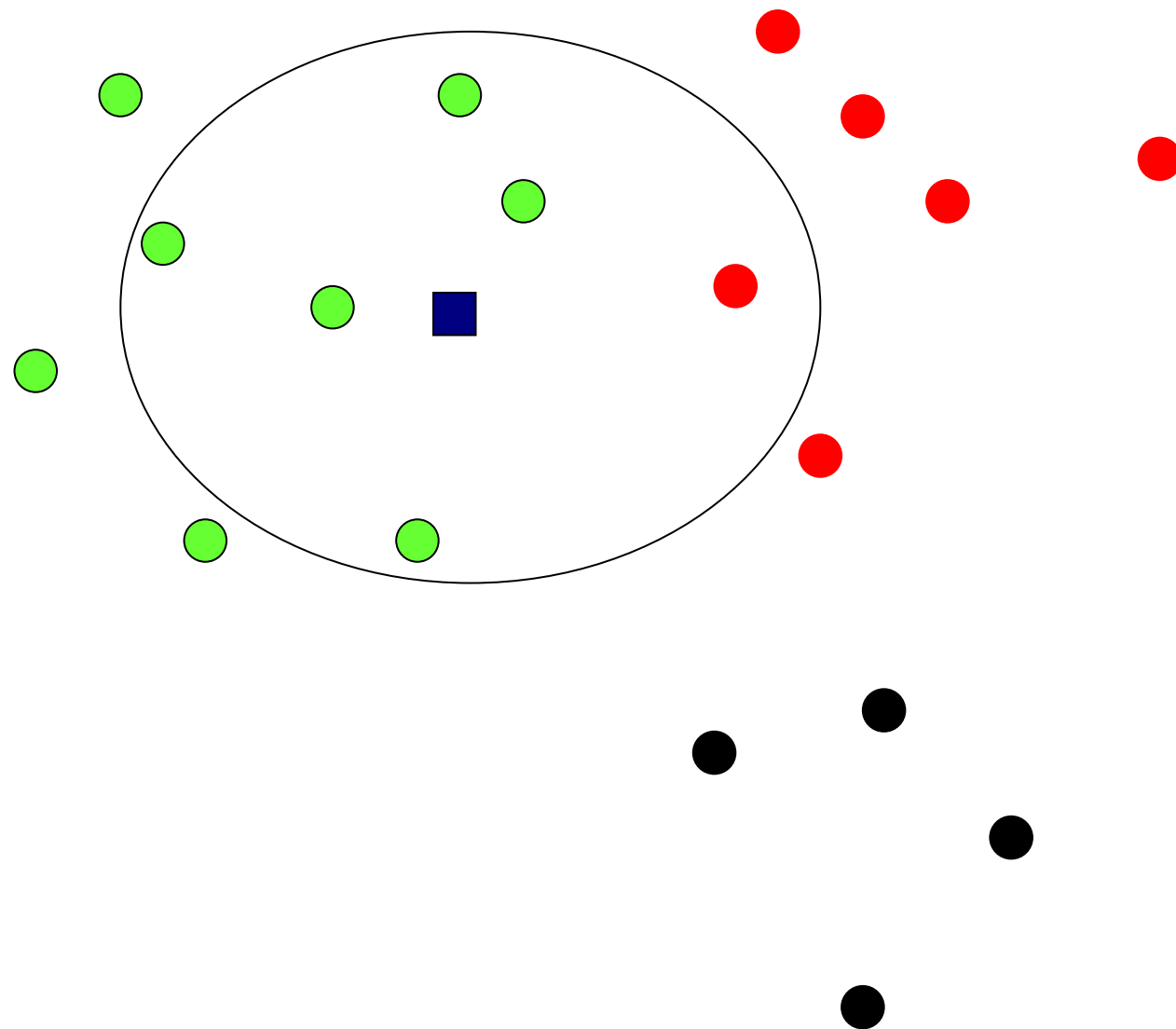
K-NN and irrelevant features



K-NN and irrelevant features



Example: $k=6$ (6NN)



● Government

● Science

● Arts

A new point ■
 $\text{Pr}(\text{science} | \blacksquare)?$

Discussions

- kNN can deal with complex and arbitrary decision boundaries.
- Despite its simplicity, researchers have shown that the classification accuracy of kNN can be quite strong and in many cases as accurate as those elaborated methods.
- kNN is slow at the classification time
- kNN does not produce an understandable model

KNN Based intrusion detection

■ Building normal behavioral model

- Calculate the distances between each test vector $\mathbf{t}$ and each vector in the training data set by using Euclidean distance:

$$dis_{eu}(\mathbf{t}, \mathbf{x}_j) = \|\mathbf{t} - \mathbf{x}_j\| = \sqrt{\sum_{i=1}^M (t_i - x_{ij})^2}$$

- Sort the distance and choose the k nearest neighbors.
- Average the k closest distance scores as the *anomaly index*.

■ Detection

- If the *anomaly index* of a test sequence vector $\mathbf{t}$ is above a threshold ε
 - the test sequence is then classified as abnormal.
 - otherwise it is considered as normal.

KNN based intrusion identification

Define normal and individual attack data sets as $D_1, D_2, \dots, D_l$;

Identification:

For each test vector **t** **do**

Calculate $dis_{eu}(\mathbf{t}, \mathbf{x}_j)$ for $\mathbf{x}_j$ in each training set;

1. 计算样本与训练集各样本的距离

2. 找到最近的k个邻居

Find k smallest scores of $dis_{eu}(\mathbf{t}, \mathbf{x}_j)$ as k -nearest neighbors;

If more than a half of k nearest neighbors correspond to a specific attack type A_k **then**

3. 如果超过k/2个属于 A_k 类，则认为属于 A_k

t is identified as A_k

Else If the number of smallest distance that corresponds to an attack type A_p is greater than those of others **then**

4. 否则选最大的 A_p 作为类别

t is identified as A_p

Else then

t is identified as a new attack

5. 都不满足就是新类型

End If

End For

KNN Based intrusion detection

Attack type	Attack category	Identification Rate (%)		
		k NN		
		$k=5$	$k=7$	$k=9$
guess_passwd	R2L	92.3	92.3	92.3
warezclient	R2L	100	100	100
warezmaster	R2L	80	80	80
back	DoS	98.5	99.5	98
neptune	DoS	99.8	99.8	97.7
pod	DoS	100	96.9	100
smurf	DoS	100	100	100
teardrop	DoS	97.7	99.4	97.8
buffer overflow	U2R	80	80	80
ipsweep	Probe	97.6	99.2	97.6
nmap	Probe	12.9	12.9	12.9
portsweep	Probe	100	100	100
satan	Probe	88.2	88.2	88.2